MANUAL TÉCNICO PORTAL DE FACTURACIÓN

"Conexión entre Base de Datos, Programación e Interfaz de Usuario"

RURANSA
Área de Sistemas
Logiran S.A.

Índice

1. Esquema y Organización de la Base de Datos 3
2. Relaciones y Estructuras de la Base de datos 3
2.1. Tabla <factura_detalle> y relaciones 3
2.2. Tabla <gb_usuarios> y relaciones 6
2.3. Tabla <cecos> y relaciones 8
2.4. Tabla <cuentas_contables> y relaciones 8
2.5. Diagrama ER del flujo de facturas 9
3. Aplicación del Patrón MVC en el Desarrollo del Portal de Facturación 9
3.1 Modelo 9
3.1.1. Características del Modelo 10
3.2. Controlador 11
3.2.1. Características del Controlador 11
3.3. Vista 12
3.3.1. Características de la Vista 13
4. Flujo General del MVC 13

1. Esquema y Organización de la Base de Datos

Las aplicaciones web están basadas en el modelo Cliente Servidor, en el cual se distribuyen las tareas entre los proveedores de recursos o servicios, llamados "Servidores" y los llamados "Clientes" que son conectados por (browser o navegadores) y utilizados por usuarios. En la siguiente imagen se muestra la arquitectura para consultar los diferentes servicios de la aplicación.

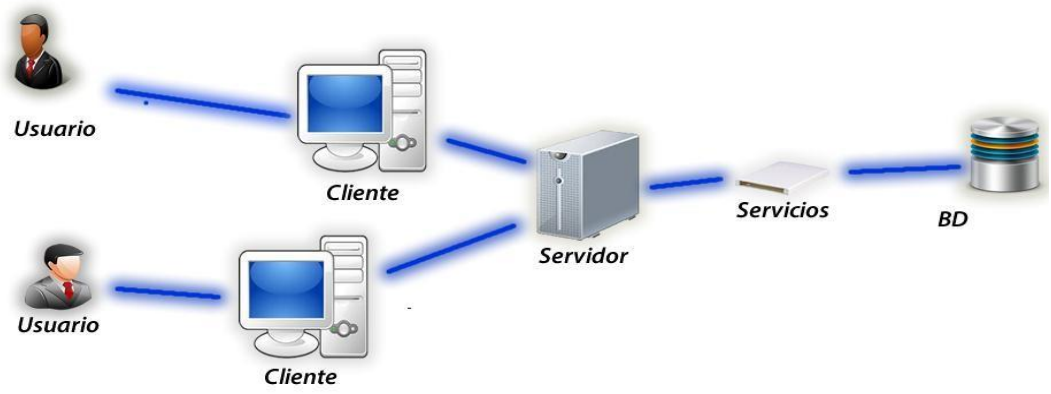

Ilustración 1. Modelo Cliente-Servidor

2. Relaciones y Estructuras de la Base de datos

2.1. Tabla <factura_detalle> y relaciones

La tabla *factura_detalle* está diseñada para almacenar información sobre los ítems o componentes individuales que conforman una factura.

- **id_factura_detalle**: Este campo funciona como un identificador único para cada elemento individual dentro de una factura.
- **razon_social**: Este campo almacena el nombre o razón social de la entidad a la que se le está facturando.
- **orden_compra**: Aquí se guarda el número o referencia de la orden de compra que originó la factura.
- **factura**: Este campo almacena el número o identificador de la factura principal a la que pertenece este detalle.
- **ceco**: Este campo registra el centro de costos al que se imputa el gasto o ítem facturado.
- **concepto**: Este campo describe el producto, servicio o concepto que se está facturando.
- **id_factura**: Este campo es una clave foránea que relaciona este detalle con la factura principal en la tabla *factura*.
- **archivos**: Este campo contiene la ruta a archivos adjuntos relacionados con el detalle de la factura.
- **id_user_aprueba**: Este campo almacena el identificador del usuario que aprobó este detalle de factura.
- **id_user_autoriza**: Este campo guarda el identificador del usuario que autorizó este detalle de factura.
- **id_user_contable**: Este campo contiene el identificador del usuario del departamento de contabilidad relacionado con este detalle.
- **estado**: Este campo indica el estado actual del detalle de la factura (por ejemplo Aprobada, Autorizada, Contabilizada, etc.).
- **num_autorizacion**: Este campo guarda un número de autorización relacionado con el detalle.
- **tipo_gasto**: Este campo clasifica el tipo de gasto asociado al detalle.
- **cuenta_contable**: Este campo registra la cuenta contable a la que se imputa el gasto.
- **id_user_gg**: Este campo almacena el identificador de un usuario relacionado con la gerencia general.
- **fecha_aprueba**: Este campo registra la fecha en que se aprobó el detalle.
- **fecha_autoriza**: Este campo guarda la fecha en que se autorizó el detalle.
- **fecha_contable**: Este campo indica la fecha en que se realizó el registro contable del detalle.
- **fecha_gg**: Este campo registra una fecha relacionada con la gerencia general.
- **id_sap**: Este campo contiene un identificador del sistema SAP en caso de que se haya contabilizado.
- **fecha_fact**: Este campo registra la fecha de la factura principal.
- **id_user_rechaza**: Este campo almacena el identificador del usuario que rechazó el detalle.
- **fecha_rechaza**: Este campo registra la fecha en que se rechazó el detalle.
- **fecha_rectificacion**: Este campo indica la fecha de rectificación del detalle.
- **fecha_anulada**: Este campo registra la fecha en que se anuló el detalle.

Ilustración 2. Entidades de la tabla factura_detalle

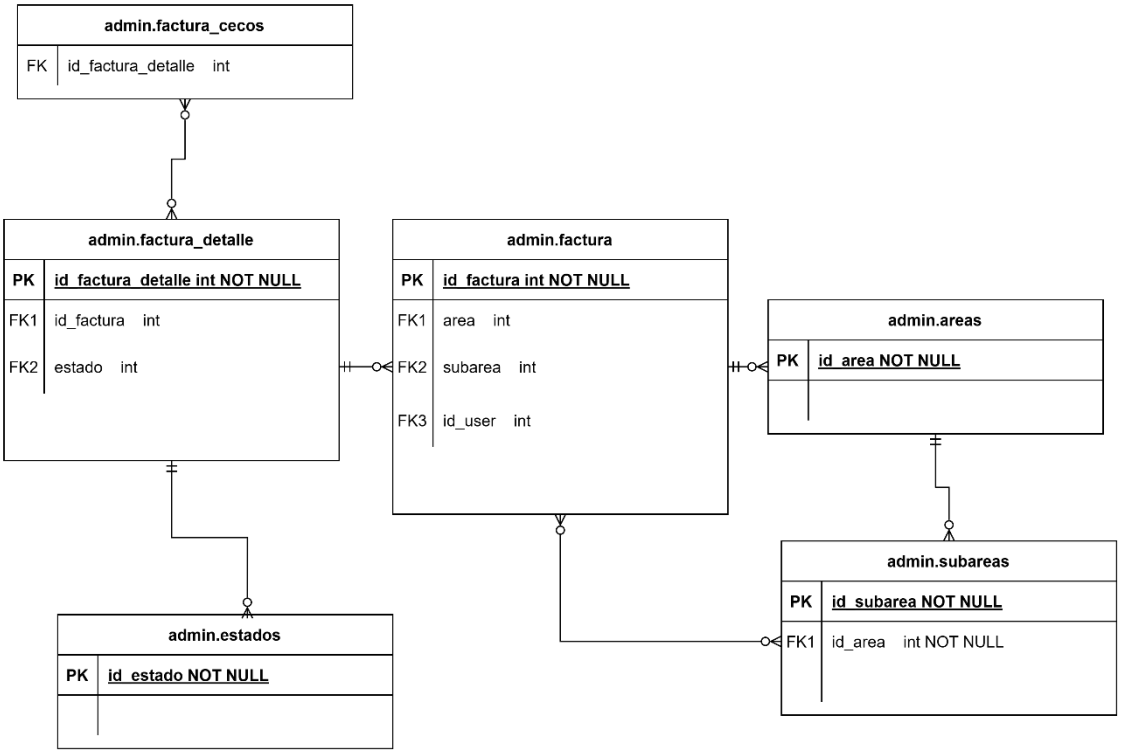

Ilustración 3. Relación con la tabla factura, area, subareas y estados

2.2. Tabla <gb_usuarios> y relaciones

La *tabla gb_usuarios* almacena la información de los usuarios del sistema.

- El campo **gb_id** funciona como identificador único de cada usuario.
- Los campos **gb_nombre**, **gb_usuario**, **gb_email** y **gb_clave** almacenan el nombre completo, el nombre de usuario que usaremos para autentificarnos, la dirección de correo electrónico y la contraseña del usuario respectivamente.
- El campo **gb_id_perfil** permite relacionar cada usuario con un perfil específico dentro del sistema (tabla *gb_perfiles*).
- Los campos **gb_fecha_registro** y **gb_hora_registro** registran la fecha y hora en que el usuario fue creado.
- El campo **gb_estatus** indica el estado actual del usuario (1-activo, 0-inactivo).
- Los campos **cedula** y **firma** almacenan el número de identificación y la firma digital del usuario en formato base64.
- La **fecha_edicion** registra la última fecha en que se modificó la información del usuario, y **id_user_edit** guarda el identificador del usuario que realizó la última modificación.
- Los campos **id_user**, **cd**, **supervisor**, **subarea** y **area** nos brindan información adicional asociada al usuario, como su identificador interno, ciudad, supervisor asignado, la subárea y área a la que pertenece.

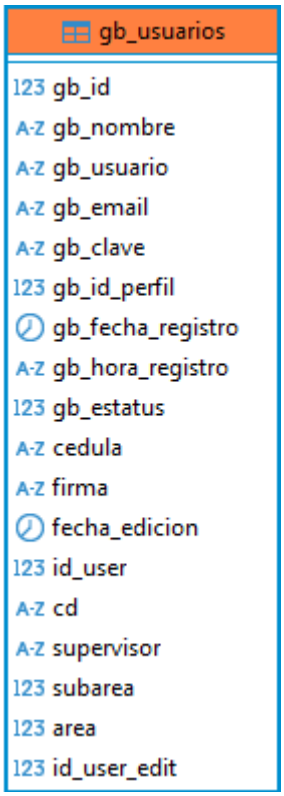

Ilustración 4. Entidades de tabla gb_usuarios

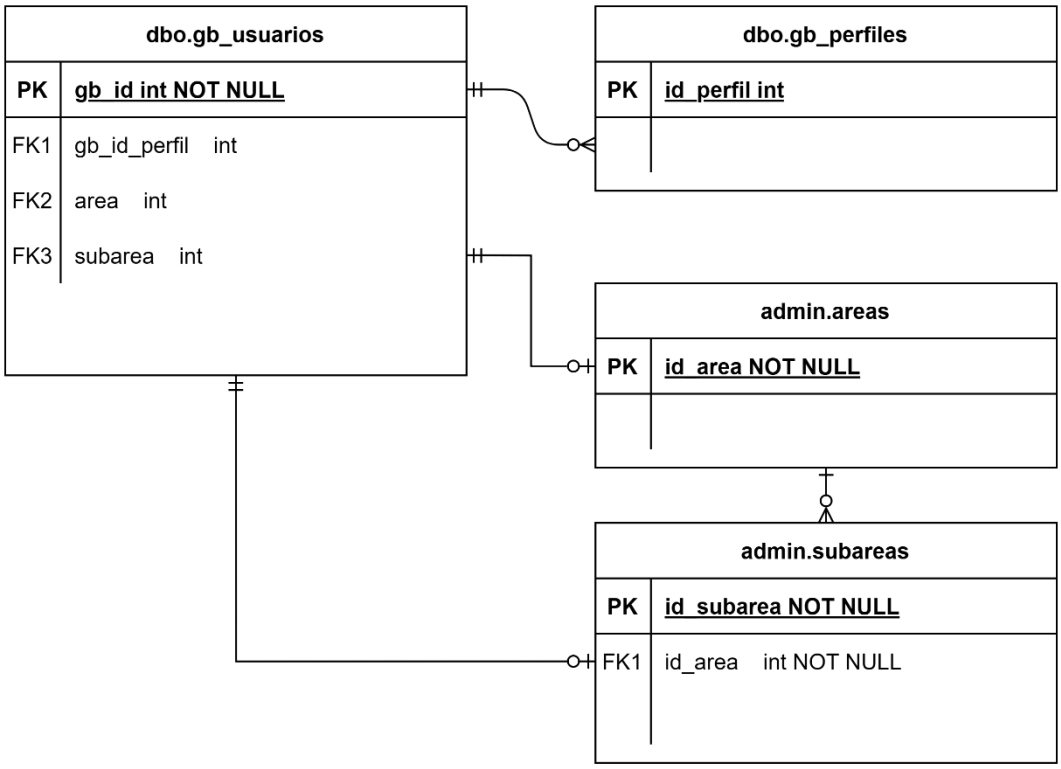

Ilustración 5. Relacion con la tabla gb_perfiles

2.3. Tabla <cecos> y relaciones

La tabla *cecos* almacena información sobre los centros de costos (CECOs) utilizados en la empresa para la gestión de gastos y presupuestos.

- **id_ceco**: Clave primaria única que identifica cada centro de costos.
- **ceco**: Código o nombre corto del centro de costos.
- **cd**: Ciudad relacionada con el centro de costos.
- **empresa**: Nombre de la empresa a la que pertenece el centro de costos.
- **estado (1-0)**: Estado actual del centro de costos.

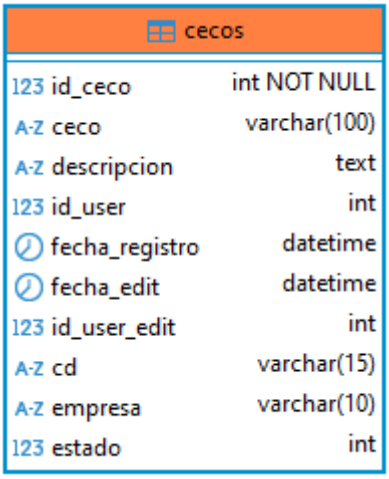

Ilustración 6. Entidades de la tabla cecos

2.4. Tabla <cuentas_contables> y relaciones

La tabla *cuentas_contables* está diseñada para almacenar información sobre las cuentas contables utilizadas en la empresa.

- **id_cuenta_contable**: Clave primaria única que identifica cada cuenta contable.
- **numcuenta**: Número o código de la cuenta contable.
- **descripcion**: Descripción detallada de la cuenta contable.
- **estado**: Estado actual de la cuenta contable (activo - 1, inactivo - 0).

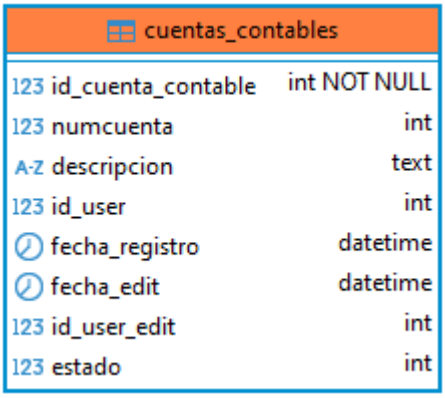

Ilustración 7. Entidades de la tabla cuentas_contables

2.5. Diagrama ER del flujo de facturas

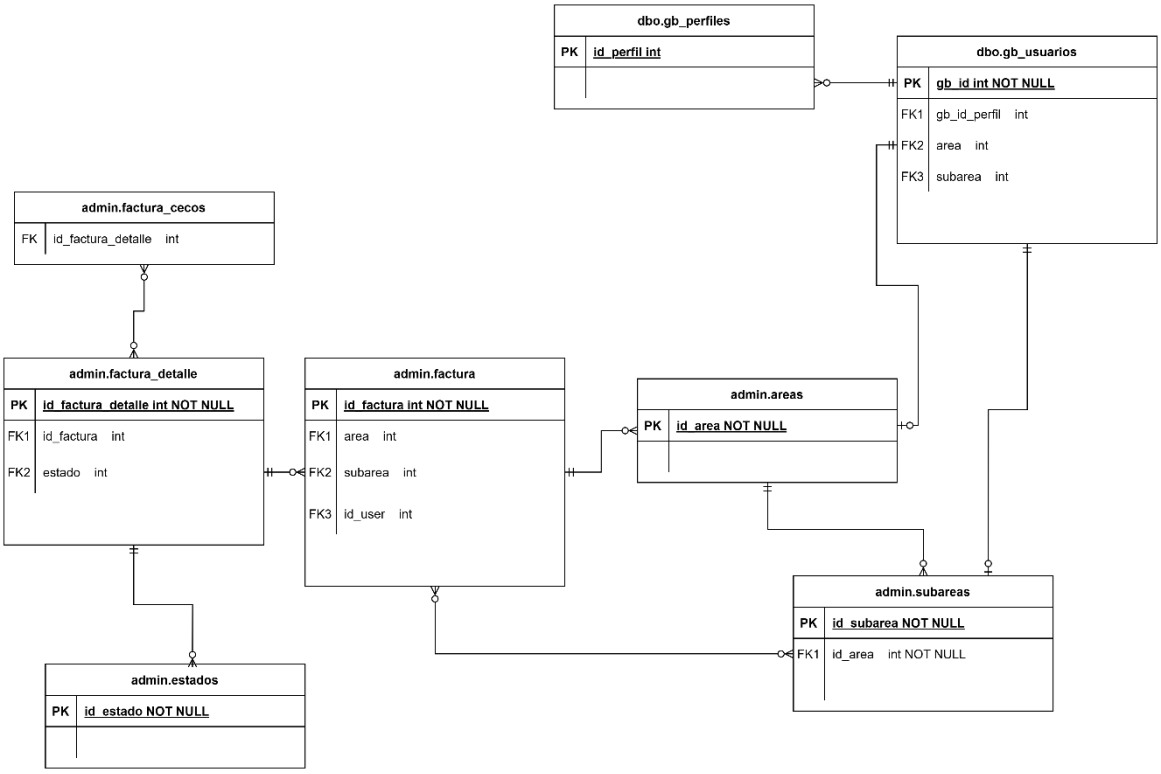

Ilustración 8. Diagrama ER completo para portal Facturación

3. Aplicación del Patrón MVC en el Desarrollo del Portal de Facturación

Este proyecto sigue el patrón de diseño Modelo-Vista-Controlador (MVC), que organiza el código en tres componentes principales: Modelo, Vista, y Controlador.

3.1 Modelo

El modelo es responsable de interactuar con la base de datos y manejar la lógica de negocio. En este proyecto, el modelo se encuentra en la siguiente ruta:

Portal-Facturacion\Model\

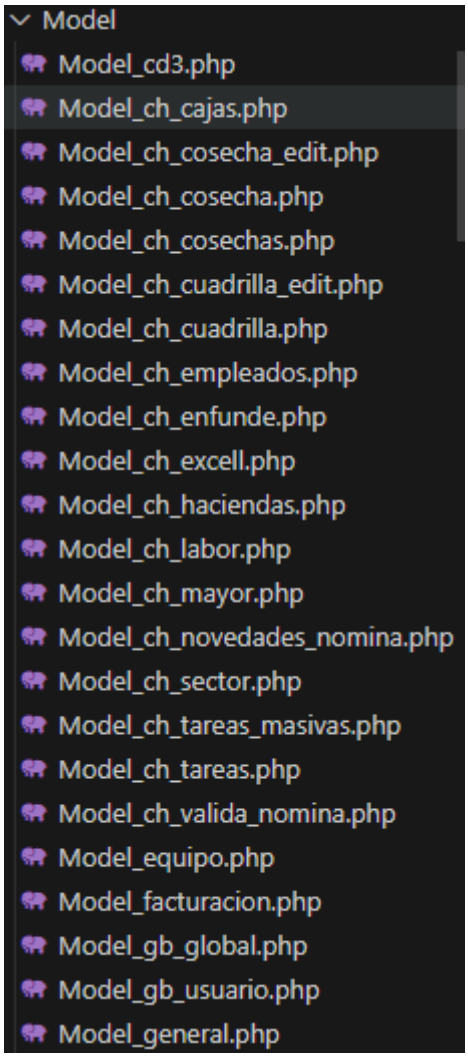

Ilustración 9. Modelos del Proyecto

3.1.1.Características del Modelo

- **Conexión a la base de datos**. El modelo utiliza una función como conexionSQL() para establecer la conexión con la base de datos (SQL Server). Donde cada función en el modelo realiza una consulta específica a la base de datos. Por ejemplo:
  - table_clientes(): Obtiene datos de la tabla admin.areas.
  - registrar_ceco($data): Inserta un nuevo registro en la tabla admin.cecos.
  - editar_ceco($data): Actualiza un registro existente en la tabla admin.cecos.
- **Estructura de datos.** Los datos obtenidos de la base de datos se procesan para luego ser almacenados en un array, que posteriormente se devuelven al controlador.

```php
<?php
public function table_clientes() {
    $conn = conexionSQL();
    $data = array();
    $sql = "SELECT * FROM admin.areas WHERE estado=1 ORDER BY area ASC";
    $resultado = sqlsrv_query($conn, $sql);
    $contador = 0;

    while ($fila = sqlsrv_fetch_array($resultado, SQLSRV_FETCH_ASSOC)) {
        $contador++;
        $data[] = $fila;
    }
                                (local variable) $contador
    $data['totalfila'] = $contador;
    return $data;
}
```

Ilustración 10. Función de ejemplo de uno de los modelos del proyecto

3.2. Controlador

El controlador actúa como intermediario entre el modelo y la vista. En este proyecto, los controladores están en la siguiente ruta:

Portal-Facturacion \Controller

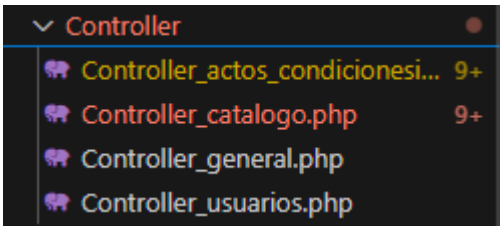

Ilustración 11. Controladores del proyecto

3.2.1.Características del Controlador

- **Procesamiento de solicitudes.** El controlador recibe las solicitudes (generalmente POST o GET) y decide qué acción realizar en función de los parámetros enviados.
- **Llamadas al modelo.** El controlador llama a las funciones del modelo para obtener o manipular datos. Por ejemplo:
    - table_estanteriaracks() llama a ModelCliente->table_estanteriaracks() para obtener datos de la base de datos.
- **Devolución de datos.** Los datos obtenidos del modelo se procesan y se devuelven al cliente (generalmente en formato JSON) para ser consumidos por el frontend.

```php
<?php
function table_estanteriaracks() {
    $Consult = new ModelCliente();
    $movements = $Consult->table_estanteriaracks($_POST['estadosearch'], $_POST['areasearch']);
    $file = array();

    for ($i = 0; $i < ($movements['totalfila']); $i++) {
        $dato['id'] = $movements[$i]['id_factura_detalle'];
        $dato['fecha_label'] = $movements[$i]['fecha_label'];
        $file[] = $dato;
    }

    $result = array('data' => $file);
    echo json_encode($result);
}
```

Ilustración 12. Ejemplo de función que maneja la lógica dentro de controlador


```
[
    02-Apr-2025 21: 58: 23 Europe/Berlin
] {
    "idchcklsttrans": "57659",
    "0": "57659",
    "idmov_recep_desp": "64925",
    "1": "64925",
    "realizadopor": "GAMBOA MEDINA FRANKLIN DENNIS",
    "2": "GAMBOA MEDINA FRANKLIN DENNIS",
    "transportista": "MIGUEL ORTIZ",
    "3": "MIGUEL ORTIZ",
    "placa": "COJ862",
    "4": "COJ862",
    "ppt": "SI",
    "5": "SI",
    "obsppt": null,
    "6": null,
    "pa": "SI",
```


Ilustración 13. Respuesta en formato JSON devuelto en respuesta a un controlador

3.3. Vista

La vista es responsable de la interfaz de usuario. En este proyecto, las vistas están implementadas principalmente en archivos JavaScript y HTML y se encuentran en las siguientes rutas:

Javascript → Portal-Facturacion \pages\global\js

HTML → Portal-Facturacion\pages\global

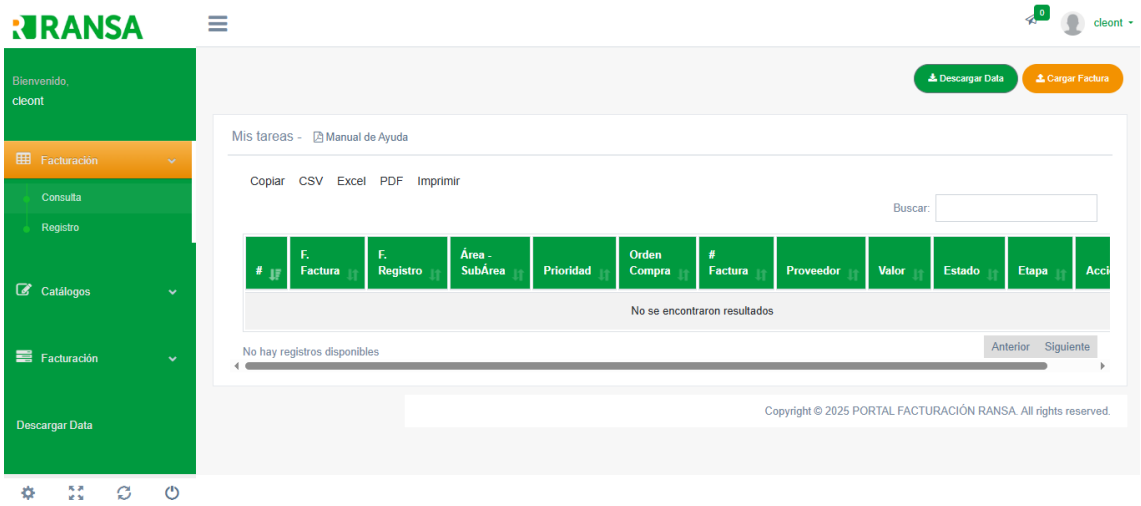

Ilustración 14. Interfaz gráfica del archivo factura

3.3.1.Características de la Vista

- **Interacción con el usuario.** La vista utiliza bibliotecas como *DataTables* para mostrar datos en tablas interactivas.
- **Solicitudes al controlador.** La vista realiza solicitudes AJAX al controlador para obtener datos. Por ejemplo, en facturas.js, la función *table_clientes* envía una solicitud POST al archivo *Controller_actos_condicionesi.php*.
- **Renderización de datos.** Los datos devueltos por el controlador (en formato JSON) se utilizan para poblar las tablas o cargar elementos de la interfaz.

4. Flujo General del MVC

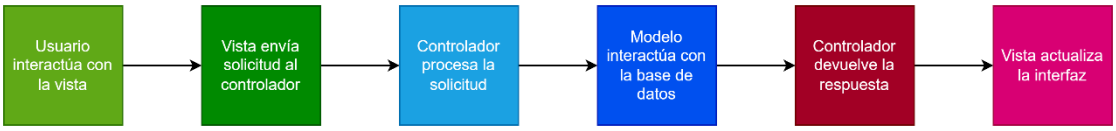

Ilustración 15. Resumen del MVC que maneja el portal de Facturación